

Update to Runcombiner

Phil Shea

1 Purpose

The MD2 Julia code output needs to be processed to create a database of measured data. The difficulty is that the system must run for many steps, very many near the critical region, before it comes to equilibrium. Previously, we defined equilibrium as a relatively low regression coefficient (records indicated $r^2 \leq 0.2$ was used correlating 1,000 step run averages). Over 50 runs of differing lengths and energies need be tracked, so some level of automation is desired. Previous work indicated that averages needed to be at least 250 steps, perhaps greater, to develop independent samples.

Digital filtering was rejected in favor of doing simple averages, as these are thermodynamic processes, and averages seem a better fit than anti-alias filtering. The three major additions to the `runcombiner` files will be spectral analysis, short term averages and re-sampling, and some form of automated regression tests attempting to choose the lengths of runs to be averaged. It is this last change which is the subject off this notebook, as some experimentation is needed to decide on the automation strategy.

1.1 Structure of the Current R Program

A run consists of multiple files which contain the statistics of a two-dimensional particle system. As of this writing, the system is 256 particles, but other sizes will be examined in the future. Each file contains a variety of data points (kinetic and potential energies, pressure, total momentum and other statistics measured at each time step. Multiple files are required because we do not know in advance how long a system at a particular energy needs to be run to achieve equilibrium. Current practice has been to initialize with 1,000 steps, and then run 5,000 more steps twice, but this is easily changed.

The multiple files for a single run will have a common filename such as “S2.0060.n.jld2”, where the `n` is a number from 0 through 9 (a limitation that can be overcome by using the alphabet following 9; in short, the filenames a simply sorted alphabetically). The database is created by calling `creatdf` with a list of files. `createdf` will figure out the root names, and then call `combineruns` with a root filename. `combineruns` creates a data frame containing all of the

run's data in order. This data frame is passed to `procMD2DF` with a parameter telling it how many initial steps to skip. `procMD2DF` then computes a linear regression, and creates statistics on each off the main columns of the data frame (there is more detail, but it is unimportant here). The main work will be in `procMD2DF`.

2 Additions

2.1 Spectral Analysis

We'll note that, while there is a very nice new R package called [gsignal](#) that replicates the [GNU Octave](#) signal package, I had spectral analysis tools already built (and really, who doesn't?), so I will use those. Previous work showed the following run to be troublesome. It looked like it might have stabilized, but it still had a high correlation coefficient even when the first 2,000 points were skipped.

```
x33 = combineruns( "H:/Data/DSP/MD/Data/1980", "S2\\.0033\\.")
plotMD2( x33)
```

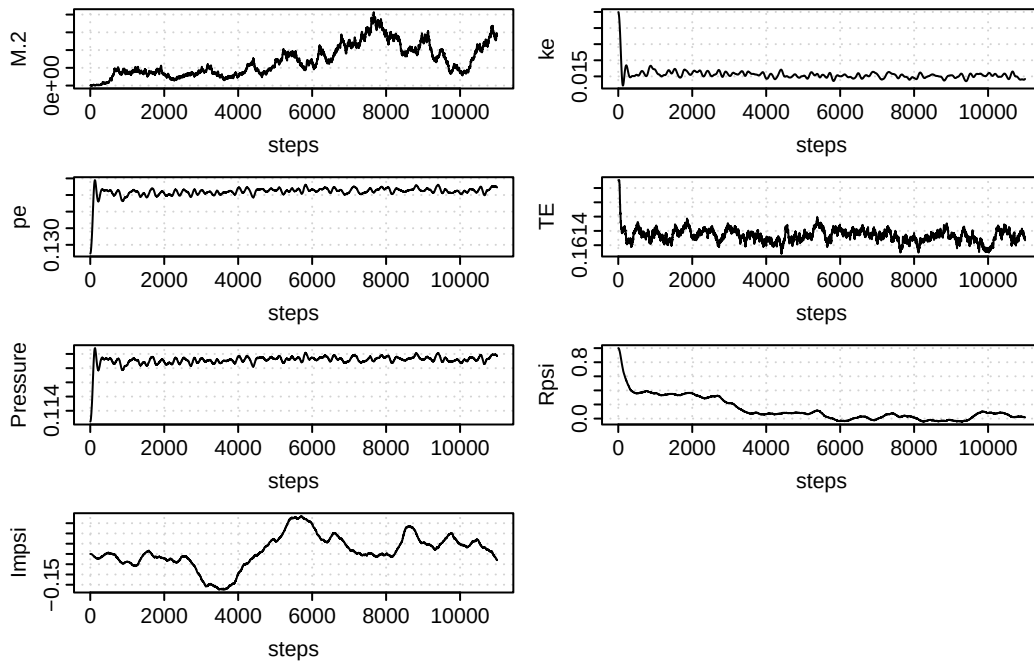


Figure 1: Plots of the basic data in run S2.0033.

Added the following code to `procMD2DF` to perform the spectral estimate:

```

ke <- if (skip == 0) df$ke else tail(df$ke, -skip)
spec <- sdf( detrend( ke), window="hanning", blocksize=fftblock,
             overlap=0.5, normalize=TRUE, prints=TRUE)
lensdf <- length( spec)
len2 <- (lensdf %/% 2)
indicies <- c( (len2 + 1):lensdf, 1:len2)
tp <- sum( spec) # total power
sdfsum <- cumsum( spec[ indicies]) / tp
width <- which.max( sdfsum > 0.995) - which.max( sdfsum > 0.005)
width <- 2 * width / fftblock # % of spectrum occupied.

```

Currently, all of the parameters to the `sdf` function (my custom function to take overlapping windowed FFTs and average the powers) with the exception of the FFT block size are currently hard coded. In my opinion, the only parameter that might need adjustment is the window. The default for the block size is 4,096 (4^6). Note that 99% of the spectrum resides in only 120 FFT bins.

Applying the above code in `procMD2DF` to `x33`, we see the following result.

```

skip <- 1000
t <- procMD2DF( x33, skip=skip)

```

sdf: Total Size: 10000, Step: 2048, Number of FFTs: 3, window Power: 4096

```
t["specwidth"] * 4096
```

```
specwidth
116
```

The spectral width of 116 indicates that any averaging and resampling should be at least 60 samples long to place the main lobe of the data in the block average. Adding this to the run statistics will help alert us to anything going awry in a run. The resemblance, perhaps a little vague, to the sum of normal variable is noted. The kinetic energy spectrum is wider and it's roll-off about (but not quite) twice as steep.

```

#op <- par( mfrow=c(1, 2))
qsdf( detrend( tail( x33$ke, -skip)), yrange=110)
qsdf( detrend( cumsum( rnorm( 10000))), yrange=110)
#par(op)

```

```
sdf: Total Size: 10000, Step: 512, Number of FFTs: 18, Window: 1000, Step: 512, Number of FFTs: 18.
```

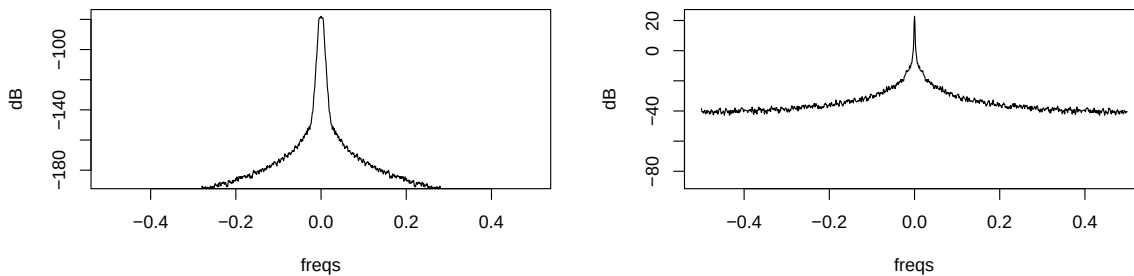


Figure 2: Temperature spectrum of runFigure 3: compared to pectrum of `detrend(cumsum( rnorm( 9000)))`

2.2 Resampling

The following simple function can perform the resampling:

```
rsmpavg <- function( x, n){
  # re-sample x by n point averages, returning one sample per average.
  # truncates if length of x is not multiple of n
  outlen <- floor( length(x) / n)
  x <- x[1:(n * outlen)]
  dim(x) <- c( n, outlen) # make x two dimensional
  apply( x, 2, mean) # essentially, column means
}
```

The function does no zero padding (we only want real measurements) and so throws out any data that is not a multiple of the averaging length. Here is the application to `x33`:

```
navg <- 500
rx33ke <- rsmavg( tail( x33$ke, -skip), navg)
rx33step <- seq( from=navg/2, by=navg, along.with=rx33ke)
rx33 <- data.frame( ke=rx33ke, step=rx33step)
plot( tail( x33$ke, -skip), type='l', panel.first=grid())
points(ke ~ step, data=rx33, col='red')
```

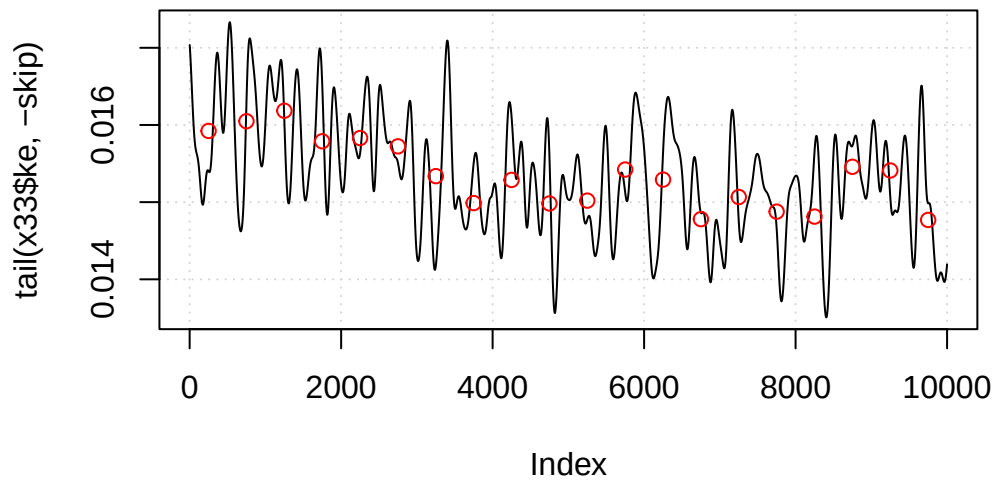


Figure 4: S2.0033 (Figure 1) kinetic energy (less the first 2,000 points) with 500 point sample averages plotted in the center of each 500 point block.

A regression on the new points show correlation.

```
lmrx33 <- lm(ke ~ step, data = rx33)
(slmrx33 <- summary(lmrx33))
```

Call:

```
lm(formula = ke ~ step, data = rx33)
```

Residuals:

Min	1Q	Median	3Q	Max
-5.026e-04	-1.815e-04	4.240e-06	1.602e-04	5.375e-04

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.591e-02	1.350e-04	117.900	< 2e-16 ***
step	-1.123e-07	2.338e-08	-4.802	0.000143 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0003015 on 18 degrees of freedom
Multiple R-squared: 0.5616, Adjusted R-squared: 0.5373
F-statistic: 23.06 on 1 and 18 DF, p-value: 0.0001427

2.3 Finding Equilibrium

How about the last two thirds of the resulting sequence?

```
lenrx33 <- nrow( rx33)
third <- floor(lenrx33 / 3) # we'll exclude the least number of points
trx33 <- tail( rx33, -third)
sstrx33 <- stat.summary( trx33$ke)
lmtrx33 <- lm( ke ~ step, data=trx33)
summary( lmtrx33)
```

Call:

```
lm(formula = ke ~ step, data = trx33)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.225e-04	-1.975e-04	-6.947e-05	1.754e-04	3.971e-04

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.524e-02	2.344e-04	65.016	<2e-16 ***
step	-2.056e-08	3.445e-08	-0.597	0.562

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0002598 on 12 degrees of freedom
Multiple R-squared: 0.02883, Adjusted R-squared: -0.0521
F-statistic: 0.3563 on 1 and 12 DF, p-value: 0.5617

```
ftlmx33 <- lm( ke ~ step, data=head( rx33, third))
plot( tail( x33$ke, -skip), type='l', panel.first=grid())
points(ke ~ step, data=rx33, col='darkred')
abline( coef=coef( lmr33), col='darkred')
points(ke ~ step, data=trx33, col='blue')
abline( coef=coef( lmtrx33), col='blue')
```

```
abline( h=sstrx33["mean"], col='darkgreen')
abline( h=sstrx33["min"], col="darkgreen", lty=2)
abline( h=sstrx33["max"], col="darkgreen", lty=2)
abline( coef=coef( ftlrx33))
```

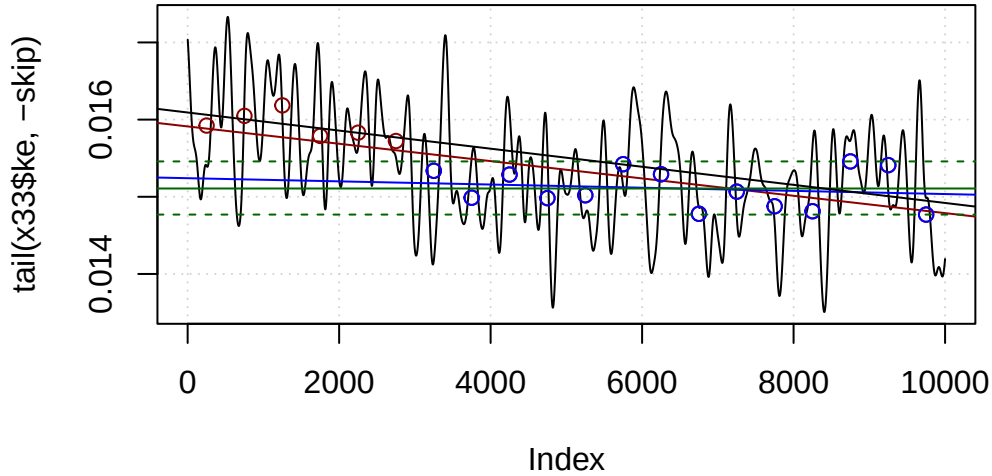


Figure 5: Fits to the resampled data, the red line is the fit to all of the data (red and blue circles), and the last two-thirds of truncated run S2-0033 (blue circles and line). Green lines indicate the mean (solid), and min & max (dashed) of the blue points. The black line is the regression line for the red points.

That is encouraging. Note that the estimate of the **step** coefficient is about half its standard error (this is the t value). The F statistic of 0.36 indicates this could be the result of a random correlation. Looking at this data, we may have been pretty lucky. It was really just dumb luck that the last point excluded was also the last point outside the min-max range. Nevertheless, a possible algorithm is forseen.

1. Skip the first 1,000 points as a general rule.
2. Re-sample the kinetic energy (ke) column.
3. Regress the re-sampled column, and check if the F statistic p-value is below 50% (that is, if the data were truly random and uncorrelated, there is a 50% chance that we would reject the large the data set and and move to the next step). If it passes the F-test, accept the data as uncorrelated, and compute the averages for the rest of the columns.

4. Having failed the first F-test, take the last two-thirds of the data, and repeat the F-test. If this fails, then a longer run is needed (i.e., we require that equilibrium be at least twice as long as the approach to equilibrium). At this point, simply report the full record's averages, and let the statistics indicate the record's problem.
5. If the second F-test passes, then compute the mean, min, and max of the two-thirds data, and also compute the regression of the first third. This second regression will give the direction of the temperature movement.
6. Find the last point outside the min or max of the two-thirds data. If the second regression has a positive coefficient (i.e., the ke is rising), use the min, and use the max otherwise.
7. Perform a third regression on the new set of points. If the F-test fails, then resort to the original two-thirds data. If the use the F-test passes, use the expanded data set for the averages.

The above has been encapsulated in `findlmlen` below. It is complicated by the attempt to extend the fit to the left.

```
findlmlen <- function( df, minp = 0.50) {
  kefit <- lmMD2( df)
  lendf <- nrow( df)
  if (kefit[ 'Fpvalue'] < minp) { # The data appears to be correlated
    third <- floor( lendf / 3) # Errs on the side of a longer 2/3s.
    ttdf <- tail( df, -third) # two thirds df
    kefit2 <- lmMD2( ttdf)
    if (kefit2[ 'Fpvalue'] >= minp) { # The 2nd 2/3s passed
      # Try to expand fit
      ftdf <- head( df, third) #first third df
      approach <- lmMD2( ftdf) # fit the approach to equilibrium
      if (sign( approach[ 'Estimate']) > 0) { # ke was growing
        adder <- which.max( ftdf$ke > min( ttdf$ke))
      } else {
        adder <- which.max( ftdf$ke < max( ttdf$ke))
      } # if (sign( approach...))
      newlen <- lendf - third + adder
      kefit3 <- lmMD2( tail( df, newlen))
      if (kefit3[ 'Fpvalue'] >= minp) return( kefit3) else
        return( kefit2)
      # the above `if` should return either way - no way to get here.
    } # if (kefit2....)
    # if we get here, then neither whole nor 2/3's passed.
  } # if kefit...
  # if we get here, then either the original fit passed the F-test,
  # or neither passed, so the original fit is returned.
  # The F p-value will show that this record is suspect.
}
```

```

    return( kefit)
}

```

Here is the result of running on the rx33 data frame:

```
findlmlen( rx33)
```

	nr	Estimate	Std.Error	r.squared	Fval
	1.400000e+01	-2.056113e-08	3.444646e-08	2.883479e-02	3.562910e-01
	Fpvalue	df.dendf			
	5.616665e-01	1.200000e+01			

These are the same results we obtained by hand (i.e., the fit was not extended).

The code below (which take about 10 seconds to run) will count the various results one might get with truly random (that is, truly uncorrelated) data.

```

len <- 50
(len3 <- floor(len/3))

```

```
[1] 16
```

```

trials <- 10000
q <- t( # need to transpose the result of replicate
  replicate( trials, findlmlen( data.frame( step = 1:len, ke = rnorm( len))))))
head( q, 10)

```

	nr	Estimate	Std.Error	r.squared	Fval	Fpvalue	df.dendf
[1,]	50	0.001826965	0.010007961	0.0006937875	0.03332492	0.85591906	48
[2,]	50	-0.009065329	0.010441484	0.0154608888	0.75377672	0.38960332	48
[3,]	50	-0.015764174	0.009354289	0.0558618371	2.84001673	0.09843489	48
[4,]	35	-0.011407089	0.017757263	0.0123505544	0.41266494	0.52505811	33
[5,]	35	0.012853790	0.019263785	0.0133120528	0.44522460	0.50925081	33
[6,]	35	0.005413027	0.016392187	0.0032935174	0.10904522	0.74332108	33
[7,]	35	-0.009536334	0.014499518	0.0129385631	0.43256941	0.51529276	33
[8,]	50	-0.002132477	0.009436287	0.0010628285	0.05107005	0.82217119	48
[9,]	50	0.008596410	0.009444569	0.0169666807	0.82845683	0.36726727	48
[10,]	35	-0.007872873	0.017316442	0.0062247705	0.20670411	0.65233980	33

The results of that are below.

```

fails <- q[, 'nr'] == len & q[, 'Fpvalue'] < 0.5
flp <- q[, 'nr'] == len & q[, 'Fpvalue'] >= 0.5 # full length pass
sp <- q[, 'nr'] < len # short pass
cat( "Fails: ", sum( fails), '\n')

```

Fails: 2762

```

cat( "Full Length Passes: ", sum( flp), '\n')

```

Full Length Passes: 5007

```

cat( "Shortened passes: ", sum( sp), '\n')

```

Shortened passes: 2231

```

cat( "Exactly Two-Thirds: ", sum( q[, "nr"] == (len - len3)), '\n')

```

Exactly Two-Thirds: 393

```

shorts <- q[, 'Fpvalue'] >= 0.5 & q[, 'nr'] < len
cat( "Average shorter length: ", mean(q[ shorts, "nr"]), "\n" )

```

Average shorter length: 34.87853

```

cat( "Max short pass: ", max( q[ sp, ]))

```

Max short pass: 38

The full correlation should fail about 50% of the time. A rather large amount of variability has been observed, but the procedure to revert to the last two-thirds adds perhaps half again as many successes. The plot below opens an interesting discussion.

```

plot( abs(Estimate) ~ Std.Error, data=q[fails, ], xlim=c(0, max( q[, 'Std.Error'])),
      ylim = c(0, max( q[, 'Estimate'])))
grid()
abline(a=0, b=1, col='red')
points( abs(Estimate) ~ Std.Error, data=q[ sp, ], col='blue',
        pch = 2)
points( abs(Estimate) ~ Std.Error, data=q[ flp, ], col='darkgreen',
        pch = 2)

```

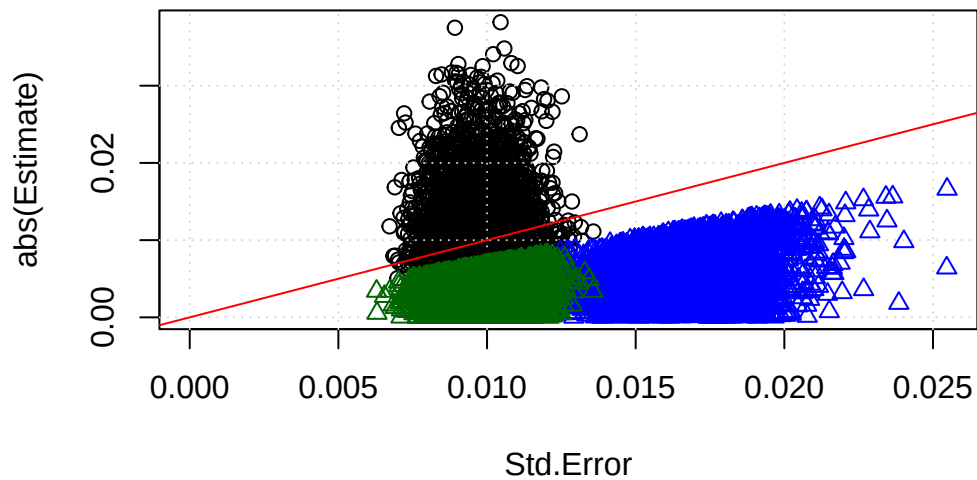
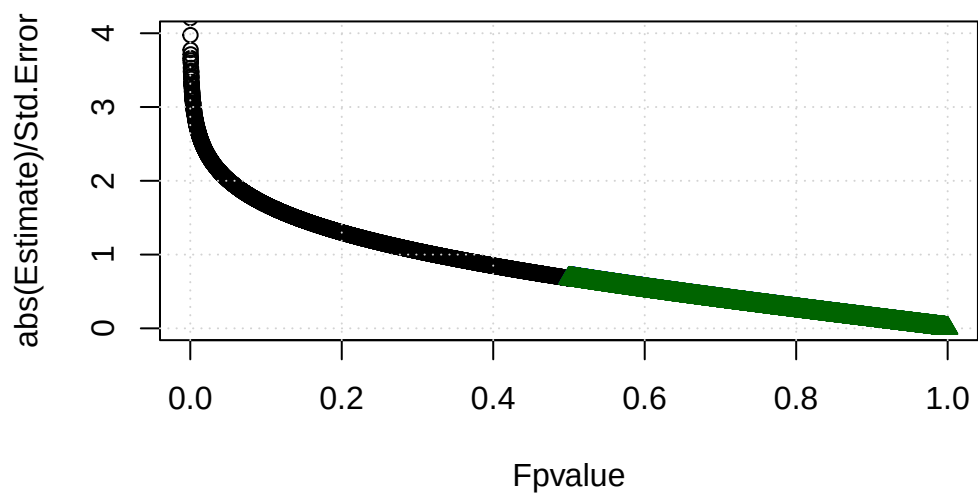
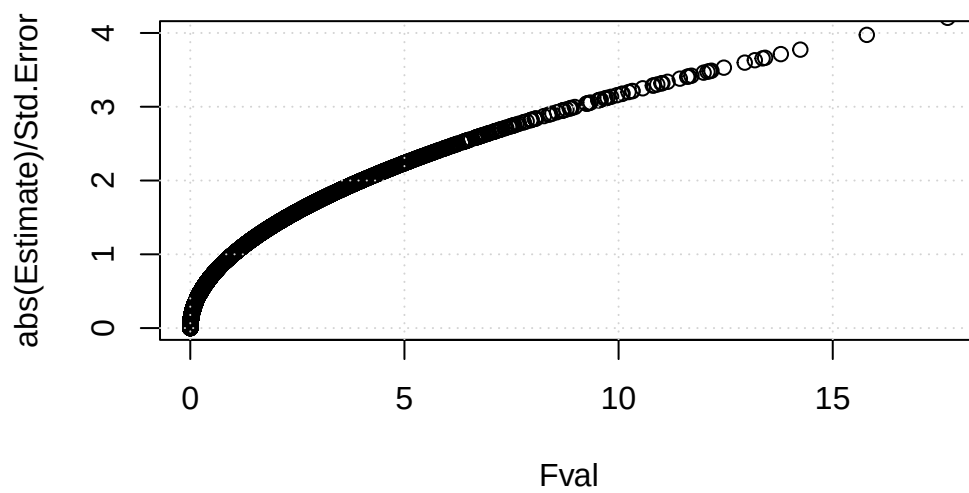


Figure 6: Absolute value of the estimate vs. the standard error. Black points are failures, green are full length passes, and blue are shortened passes. The red line shows equality (estimate equals the standard error).

```
plot( abs(Estimate)/Std.Error ~ Fpvalue, data=q[fails, ],
      xlim=c(0, max( q[, 'Fpvalue'])), ylim = c(0, 4))
grid()
points( abs(Estimate)/Std.Error ~ Fpvalue, data=q[ sp, ], col='blue',
        pch = 2)
points( abs(Estimate)/Std.Error ~ Fpvalue, data=q[ flp, ], col='darkgreen',
        pch = 2)
```



```
plot( abs(Estimate)/Std.Error ~ Fval, data=q,
      xlim=c(0, max( q[, 'Fval'])), ylim = c(0, 4))
grid()
```



3 Applying to MD2 Data

The max and mins will be gathered from the raw data, while means and standard deviations will come from the re-sampled block averages.

```
(rec <- procMD2DF( x33, skip=1000, rsblk=500))
```

sdf: Total Size: 10000, Step: 2048, Number of FFTs: 3, window Power: 4096

total.steps	steps.avg	TE.init	xMomentum.mean.rs
1.100000e+04	7.000000e+03	1.623105e-01	-2.579086e-13
xMomentum.sd.rs	yMomentum.mean.rs	yMomentum.sd.rs	M.2.mean.rs
2.060392e-11	-4.333660e-11	1.752908e-11	2.600937e-21
M.2.sd.rs	ke.mean.rs	ke.sd.rs	pe.mean.rs
1.703695e-21	1.547193e-02	4.454485e-04	1.460700e-01
pe.sd.rs	TE.mean.rs	TE.sd.rs	Pressure.mean.rs
4.315951e-04	1.615419e-01	4.994692e-05	1.211637e-01
Pressure.sd.rs	Rpsi.mean.rs	Rpsi.sd.rs	Impsi.mean.rs
2.031029e-04	1.302610e-01	1.335037e-01	5.688831e-04
Impsi.sd.rs	psiabs.mean.rs	psiabs.sd.rs	xMomentum.min.raw
9.215538e-02	1.736435e-01	1.125690e-01	-4.824918e-11
xMomentum.max.raw	yMomentum.min.raw	yMomentum.max.raw	M.2.min.raw
4.525869e-11	-7.810397e-11	2.352341e-12	5.436527e-22
M.2.max.raw	ke.min.raw	ke.max.raw	pe.min.raw
8.276650e-21	1.356339e-02	1.733110e-02	1.441962e-01
pe.max.raw	TE.min.raw	TE.max.raw	Pressure.min.raw
1.480852e-01	1.612831e-01	1.617896e-01	1.201866e-01
Pressure.max.raw	Rpsi.min.raw	Rpsi.max.raw	Impsi.min.raw
1.221598e-01	-3.710786e-02	3.664317e-01	-1.736060e-01
Impsi.max.raw	psiabs.min.raw	psiabs.max.raw	nr
1.843642e-01	1.355604e-02	3.669469e-01	1.400000e+01
Estimate	Std.Error	r.squared	Fval
-1.028056e-05	1.722323e-05	2.883479e-02	3.562910e-01
Fpvalue	df.dendf	rtv	psiavg
5.616665e-01	1.200000e+01	8.289074e-04	1.302623e-01
specwidth			
2.832031e-02			

```
rec['psiabs.max.raw']
```

```
psiabs.max.raw
0.3669469
```

```
setwd("H:/Data/DSP/MD/Data/1980")  
getwd()
```

```
[1] "H:/Data/DSP/MD/Data/1980"
```

```
files = Sys.glob( file.path( "S2.*.scalars.csv"))  
length(files)
```

```
[1] 300
```

```
MD2DF <- createdf( ".", files, clusters=num_cores)
```

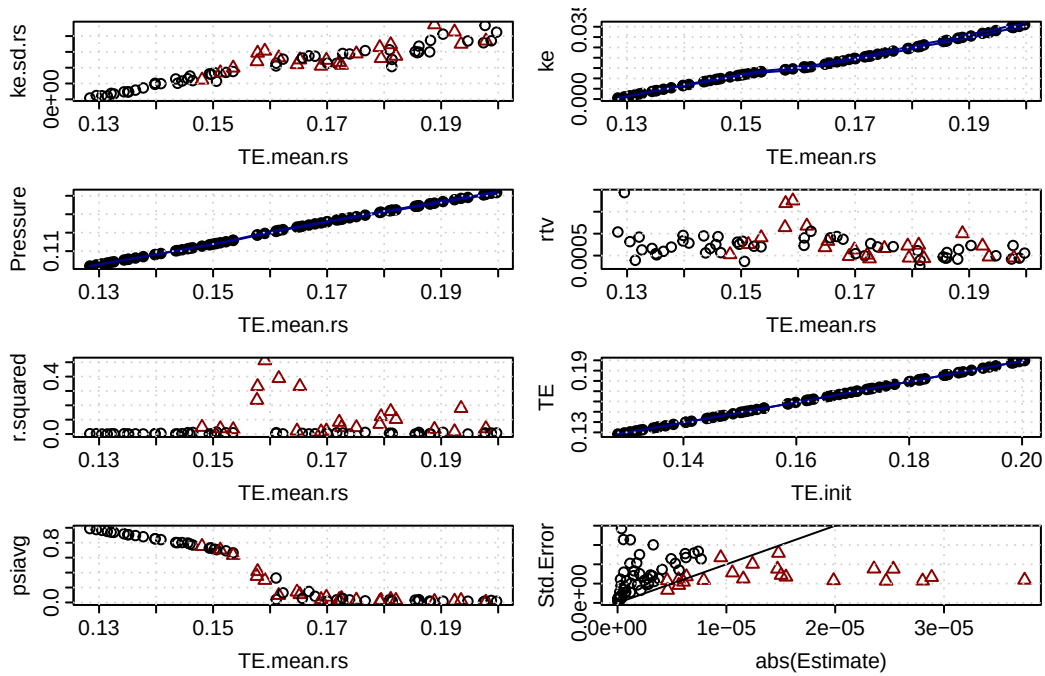
```
sdf: Total Size: 11000, Step: 2048, Number of FFTs: 4, window Power: 4096
```

```
save( MD2DF, file="MD2DF.3.S2.Rdata")
```

```
cat( "Observations: ", nrow(MD2DF),  
     ", Valid: ", sum( MD2DF[, 'Fpvalue'] >= 0.5))
```

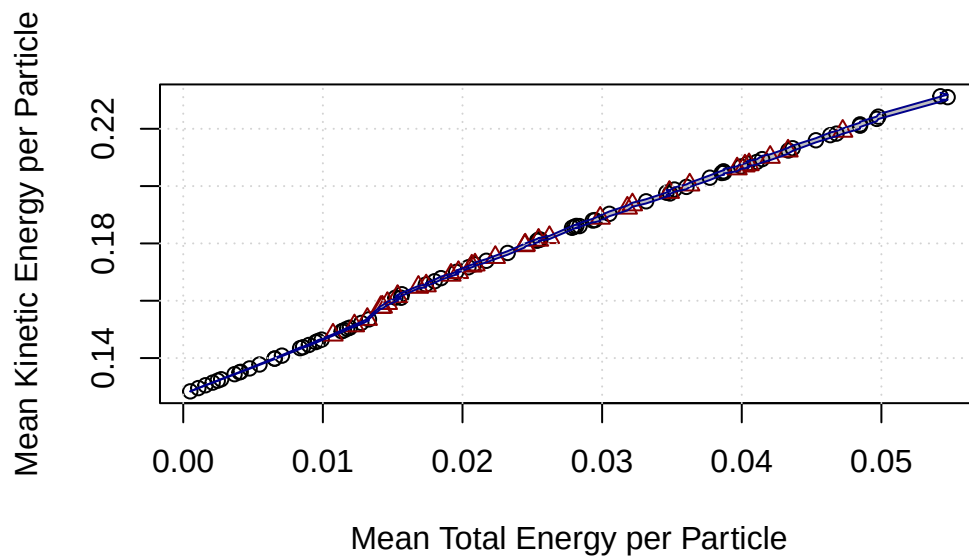
```
Observations: 100 , Valid: 71
```

```
MD2DF <- MD2DF[ order( MD2DF$TE.mean.rs),]  
op= par( mfrow = c(4, 2), mar = c(3, 3, 0.5, 0.5), tcl = -0.3,  
         mgp = c(1.7, 0.4, 0) )  
plotScalars( subset( MD2DF, TE.mean.rs< 0.2))
```



```
par(op)
```

```
plotconf( MD2DF[, 'ke.mean.rs'], MD2DF[, 'TE.mean.rs'], MD2DF[, 'ke.sd.rs'],
          MD2DF[, 'Fpvalue'], linethresh = 200, vpch=1,
          xlab="Mean Total Energy per Particle",
          ylab="Mean Kinetic Energy per Particle")
```



Re-running it with `rblk` at 500.

```
#setwd("H:/Data/DSP/MD/Data/1980")
getwd()
```

```
[1] "C:/Users/phils/Documents/DSP/MD"
```

```
files = list.files("H:/Data/DSP/MD/Data/1980", "S2.*.scalars.csv")
length(files)
```

```
[1] 300
```

```
MD2DF <- createdf( "H:/Data/DSP/MD/Data/1980", files, clusters=num_cores,
  rsblk=500)
```

```
sdf: Total Size: 11000, Step: 2048, Number of FFTs: 4, window Power: 4096
```

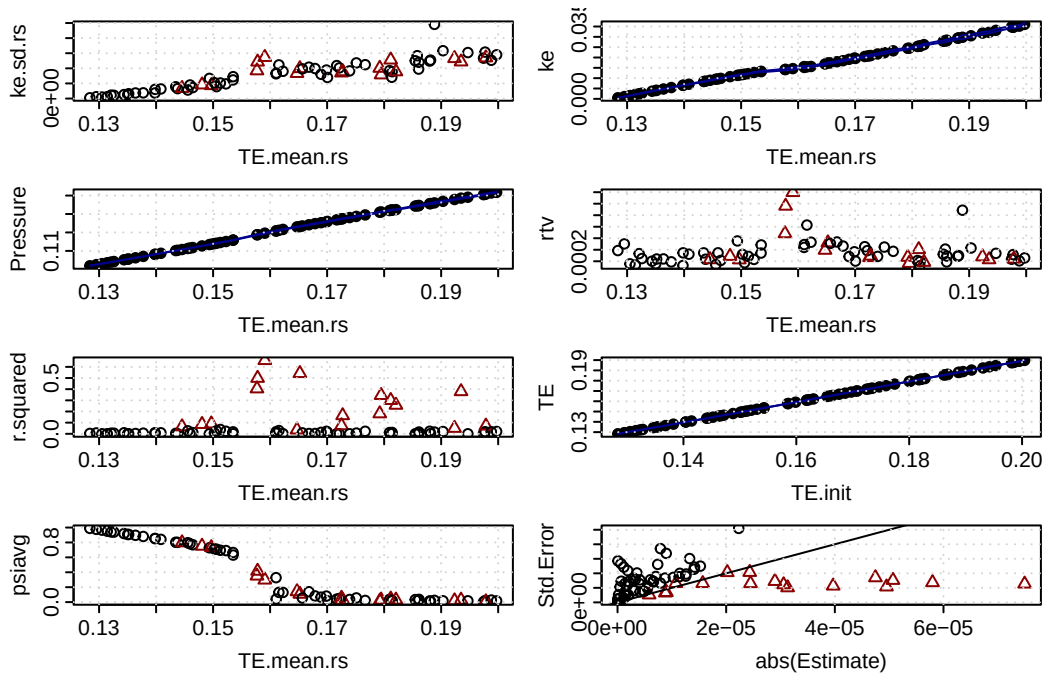
```
save( MD2DF, file="MD2DF.3.S2.Rdata")
```

These results are a little better.

```
cat( "Observations: ", nrow(MD2DF),
      ", Valid: ", sum( MD2DF[, 'Fpvalue'] >= 0.5))
```

Observations: 100 , Valid: 78

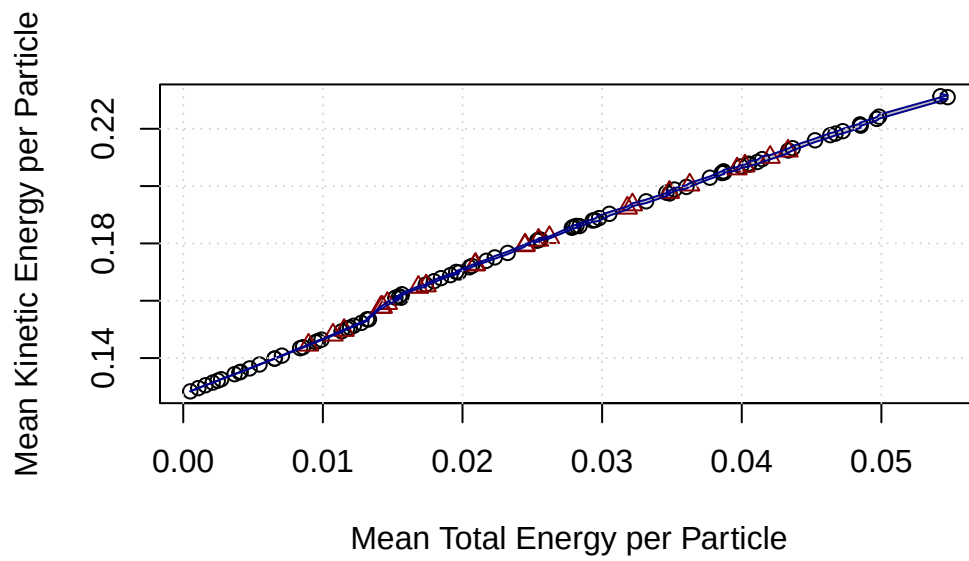
```
MD2DF <- MD2DF[ order( MD2DF$TE.mean.rs),]
op= par( mfrow = c(4, 2), mar = c(3, 3, 0.5, 0.5), tcl = -0.3,
         mgp = c(1.7, 0.4, 0) )
plotScalars( subset( MD2DF, TE.mean.rs< 0.2))
```



```
par(op)
```

Not much different here.

```
plotconf( MD2DF[, 'ke.mean.rs'], MD2DF[, 'TE.mean.rs'], MD2DF[, 'ke.sd.rs'],
          MD2DF[, 'Fpvalue'], linethresh = 200, vpch=1,
          xlab="Mean Total Energy per Particle",
          ylab="Mean Kinetic Energy per Particle")
```



```
(rerun <- with( MD2DF, {MD2DF[ Fpvalue < 0.5, c('series', 'energy')]}))
```

	series	energy
16	S2	0016
21	S2	0021
23	S2	0023
30	S2	0030
31	S2	0031
36	S2	0036
38	S2	0038
37	S2	0037
51	S2	0051
47	S2	0047
52	S2	0052
56	S2	0056
59	S2	0059
53	S2	0053
66	S2	0066
67	S2	0067
82	S2	0082
73	S2	0073
74	S2	0074

79	S2	0079
83	S2	0083
92	S2	0092

List of 22 runs to add steps to.

```
rerun$energy
```

```
[1] "0016" "0021" "0023" "0030" "0031" "0036" "0038" "0037" "0051" "0047"  
[11] "0052" "0056" "0059" "0053" "0066" "0067" "0082" "0073" "0074" "0079"  
[21] "0083" "0092"
```